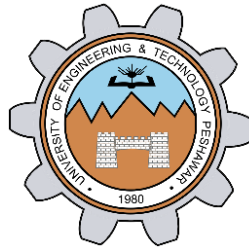


LAB # 01
INTRODUCTION TO MODELSIM AND GATE LEVEL
MODELING



Spring 2024
CSE-308L Digital System Design Lab

Submitted by: **Shahzad Bangash**

Registration No: **21PWCSE1980**

Class Section: **C**

“On my honor, as student at University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Engr. Shahzada Fahim Jan

February 25, 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives of The Lab:

- Introduction to the ModelSim software

MODELSIM:

ModelSim-Intel is a software product of intel where we design, simulate, and test our hardware in hardware descriptive language such as Verilog.

Lab Tasks:

1. Implement a buffer at the gate level.

DUT Code:

```
// Design module - module is a keyword, this is where we define our module.
module buffer(O,I);
input I; //input is keyword
output O; //output is keyword

//Two types level
//Gate level - that uses gates
//Behaviour level - coding the behaviours of gates, using &,&~,| operators;

buf b1(O,I); //buf is keyword used for buffer gate
endmodule
```

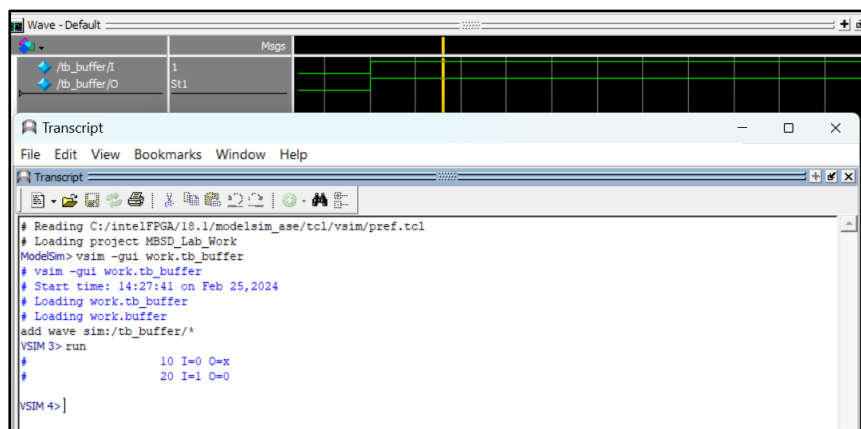
Test Bench Code:

```
// Test bench module
module tb_buffer();
reg I;
wire O;

buffer my_buffer(.O(O),.I(I)); // explicit association
initial begin
    // initial is verilog keyword used for giving and testing values
    // begin - end is used if the code is more than 1 line.
    #10; I=0; $display("%d I=%b O=%b", $time,I,O); // $time is used to print time
    #10; I=1; $display("%d I=%b O=%b", $time,I,O);

end
endmodule
```

Output:



2. Implement an inverter at the gate level.

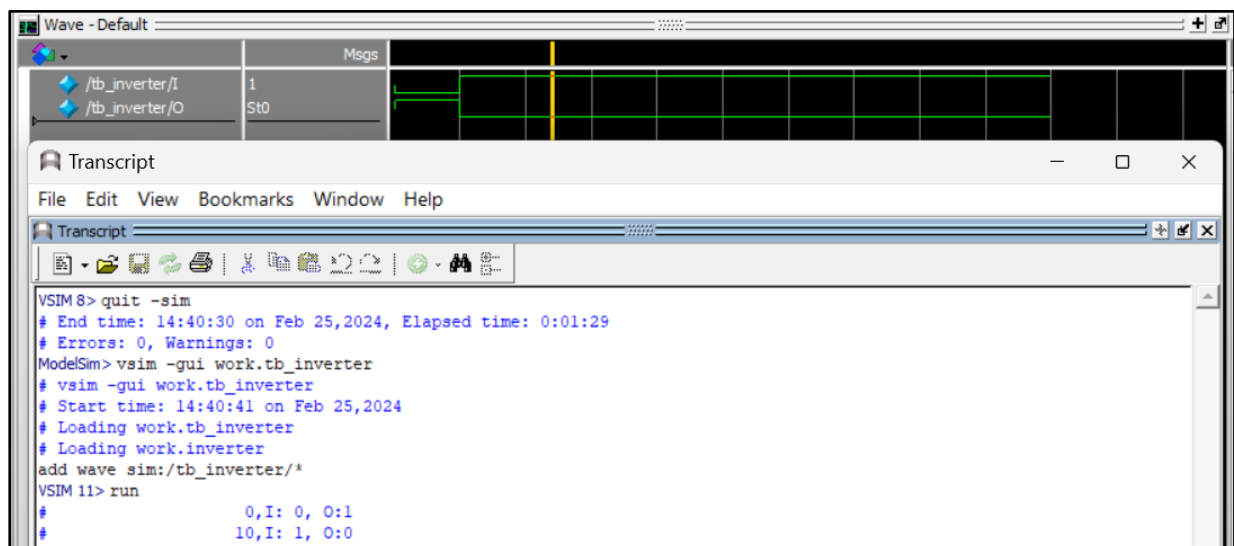
DUT Code:

```
1  module inverter(O,I) ;
2      input I;
3      output O;
4      not n1(O,I) ;
5  endmodule
```

Test Bench Code:

```
1  module tb_inverter() ;
2      reg I;
3      wire O;
4      inverter inv1(O,I) ;
5      initial begin
6          I=0;
7          #10; I=1;
8      end
9      initial
10         $monitor("%d,I: %b, O:%b",$time,I,O);
11 endmodule
```

Output:



3. Implement an OR gate using NAND gates.

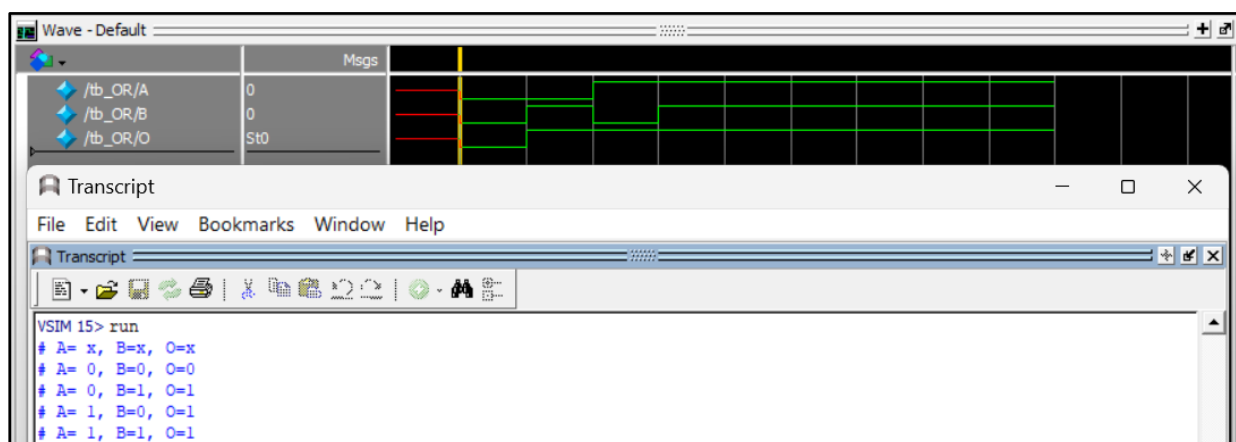
DUT Code:

```
1 // Design module
2 module OR(O,A,B);
3 input A,B;
4 output O;
5
6 wire nA,nB;
7
8 nand n1(nA,A);
9 nand n2(nB,B);
0 nand n3(O,nA,nB);
1 endmodule
```

Test Bench Code:

```
1 // Test bench module
2 module tb_OR();
3 reg A,B;
4 wire O;
5
6 OR my_OR(O,A,B);
7 initial begin
8
9     #10; A=0; B=0;
0     #10; A=0; B=1;
1     #10; A=1; B=0;
2     #10; A=1; B=1;
3 end
4
5 initial
6     $monitor("A= %b, B=%b, O=%b",A,B,O); //monitor is another way to print values onto the console
7                                     // but the it is executed whenever the value changes given to it.
8 endmodule
```

Output:



4. Implement the following equation where z is output and x1, x2, x3, x4, and x5 are inputs of the circuit.
- $$z = (y1 + y2)'$$
- $$y1 = x1.x2$$
- $$y2 = (x3.x4.x5)$$

DUT Code:

```
C:\Users\SHAHZAD BANGASH\Documents\6th Semester\DSD Lab\DSD_Lab_ModelSim_Work\Ripple_Carry_Adder.v
2  input x1,x2,x3,x4,x5;
3  output z;
4  wire y1,y2,o;
5  and a1(y1,x1,x2);
6  and a2(y2,x3,x4,x5);
7  or o1(o,y1,y2);
8  not n1(z,o);
9  endmodule
```

Test Bench Code:

```
1  module tb_lab1_task4();
2  reg x1,x2,x3,x4,x5;
3  wire z;
4  lab1_task4 t1(z,x1,x2,x3,x4,x5);
5  initial begin
6      x1=0; x2=0; x3=0; x4=0; x5=0;
7      #10; x1=1; x2=1; x3=1; x4=1; x5=1;
8  end
9  initial
10     $monitor("%d,x1:%b, x2:%b, x3:%b, x4:%b, x5:%b, z:%b,", $time,x1,x2,x3,x4,x5,z);
11
12 endmodule
```

Output:

